

Scheduled jobs

Running an SD command on a timer, and the permit list that decides which ones.

A scheduled task can run an SD command, and only the commands an administrator has named for it.

Typing a command after `sd` — `sd my.report` — needed a session started with *Run as administrator* in earlier builds of this port. A scheduled task does not run that way, so there was no way to have SD do anything on a timer without handing the job administrator rights.

There are two ways past that now:

an elevated session	runs anything, exactly as before
---------------------	----------------------------------

anything else	runs what is listed for its account in the SD system file <code>batch.jobs</code>
---------------	---

Nothing changes for a session you type at. Commands entered at the `:` prompt are unaffected, and so is plain `sd` with no command after it.

There is no password anywhere in this. The job signs in as its own Windows account and SD puts it in the matching SD account, exactly as it would for a person at a keyboard. Nothing has to store a credential for the job to use, and the job grants nobody anything — no other account is in its group.

The permit list

`batch.jobs` is a directory file in `C:\ProgramData\SD\sdsys`, so **one record per account, named after the account**, holding one command name per line.

Only an administrator can change it. It is read-only to SD users, by the same control and the same script as the `os.users` permit list — a user who could add a line to their own record would be granting themselves the command line.

It is keyed by the account you end up in, not by the Windows account name, and the two are usually the same. `sd - a<name>` cannot be used to reach somebody else's list: that form is refused unless the account is your own.

Any account can be given a list, whatever its tier — standard, programmer or administrator. The account type decides nothing here; the administrator's list does.

A standard account cannot write its own paragraph, though. `ed` is not in the standard tier, so the paragraph the job runs has to be put there by somebody who can edit — see Account types.

Both shapes of record are read — one name per line, and a multivalued field 1 — so a record written with `ed` and a record written by a program agree. A name still has to match exactly.

What may be listed

Two rules, and both are tested when the command runs rather than when it is listed:

1. A single name with nothing after it. `my.report` may be listed; `my.report yesterday` may not, and neither will run:

A command run from the command line must be a single name with nothing after it. Put anything the job needs to vary inside the paragraph or sentence being run.

That is the rule doing the security work. "*It must be in the VOC*" would be worth nothing on its own — every verb is a VOC entry — so it is the absence of arguments that stops a listed name being turned into something else.

2. A paragraph or a sentence in that account. A VOC entry of type `PA` or `S`. Anything else — a verb, a file pointer — is refused by name:

MY.REPORT has to be a paragraph or a sentence in this account to be run from the command line, and it is neither.

Setting one up

Four steps, and only the third needs an administrator.

1. Write the work as a paragraph in the account that will run it. A `PA` record in that account's VOC, with the commands on the lines after the type:

```
:ed voc my.report
```

2. Check it runs when you type it, in that account, before putting it on a timer.

3. Add the name to the account's record in `batch.jobs`, from an elevated session:

```
:logto sdsys  
:ed batch.jobs fred
```

One command name per line. Create the record if the account has none.

4. Create the scheduled task. Windows Task Scheduler runs it; two fields carry the whole of it:

Program/script	C:\Program Files\SD\usr\bin\sd.exe
Add arguments	my.report

The task runs as a Windows account, and that account needs a matching SD account — that is the whole of the sign-in. There is no password to configure anywhere in SD for it.

Do not tick "run with highest privileges". It is not needed, and the whole point of `batch.jobs` is a job that runs without administrator rights. A task that is elevated passes the gate on elevation alone and never consults the list, so it would also be a job nobody had approved a command for.

Use a Windows account you already had — that is the case the design was built around. Accounts that `create.account` makes are denied interactive logon at this machine on purpose, which Task Scheduler may or may not accept as the identity a task runs as.

When it refuses

It fails closed. Every path that does not reach the end of the check leaves the command refused, and each one writes a reason to the audit trail:

WHAT IS WRONG	RECORDED AS
the command line carried an argument	command line carried arguments
<code>batch.jobs</code> is missing or unreadable	batch.jobs could not be opened
the name is not in this account's record	command is not on the account list
the name has no VOC entry at all	no VOC record for the command
it has one, but it is not <code>PA</code> or <code>S</code>	VOC type is not PA or S

The message on the screen for the third of those names both the command and the account, and says where an administrator grants it:

```
MY.REPORT is not a command that account FRED may run from the command line.
```

An administrator grants it by adding the name to the account's record in the SD system file `batch.jobs`, which only an administrator can change.

A refusal is not a hang. `sd <command>` no longer walks into a password prompt it cannot answer — see Running SD.